Comprehensive Statistical Modeling: Student Performance

Daniel D. Mondal

2026-05-07

1. Executive Summary

This document applies a comprehensive suite of statistical modeling techniques to predict student exam_score based on the student.csv dataset. The techniques explored include Ordinary Least Squares (OLS), Subset Selection, Regularization (Ridge, LASSO, Elastic Net) and Non-Parametric modeling (KNN).

2. Environment Setup & Data Preparation

First, we load the required packages, import the dataset, and prepare our training and testing environments. Because the dataset is large ($N = 80{,}000$), we implement an 80/20 train-test split.

```
# Load required libraries
library(tidyverse)
library(glmnet)
library(pls)
library(caret)
library(car)
library(knitr)

# Set global seed for exact reproducibility
set.seed(2026)

# Load the dataset
df = read.csv("student.csv", stringsAsFactors = TRUE)

# Data Cleaning
# Remove 'student_id' as it has no predictive value and drop missing targets
df_model = df %>%
  select(-student_id) %>%
  drop_na(exam_score)

# Create Train / Test Split (80% / 20%)
train_index = createDataPartition(df_model$exam_score, p = 0.8, list = FALSE)
train_data = df_model[train_index, ]
test_data  = df_model[-train_index, ]

cat("Training Set Rows:", nrow(train_data), "\n")
```

```
## Training Set Rows: 64001
```

```
cat("Testing Set Rows:", nrow(test_data), "\n")
```

```
## Testing Set Rows: 15999
```

2.5 Exploratory Data Analysis (EDA)

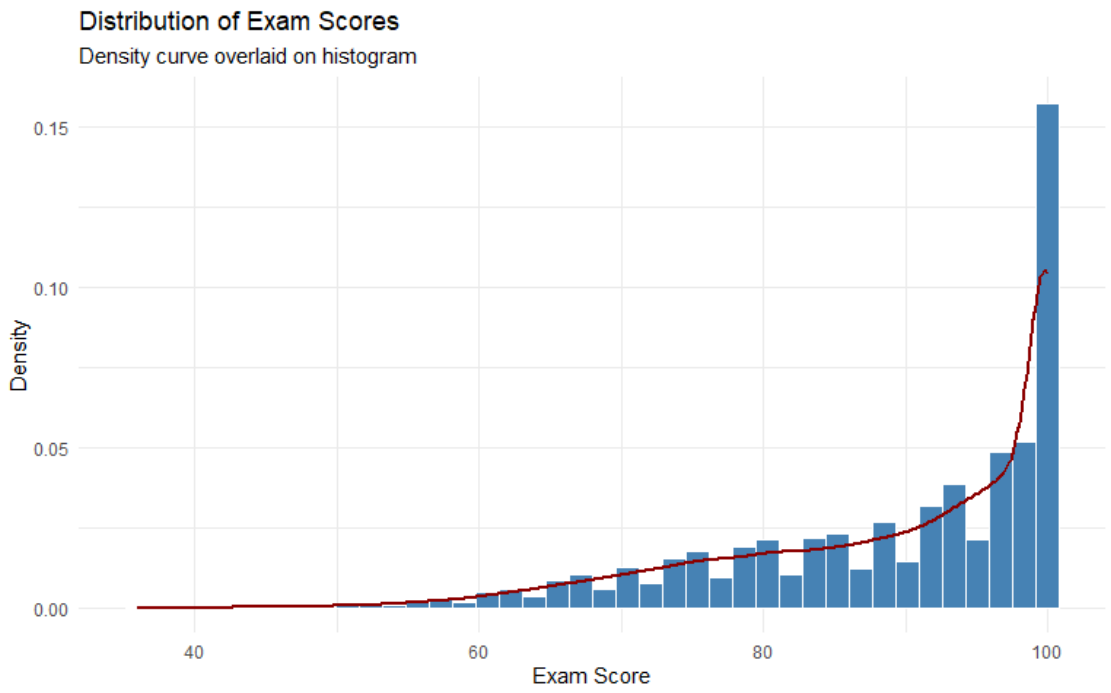
Before building predictive models, it is crucial to understand the geometry of our data, the distribution of our target variable, and the relationships between predictors.

```
# Load additional libraries for visualization
library(ggplot2)
library(corrplot)
library(gridExtra) # For arranging plots side-by-side
```

2.5.1 Target Variable Distribution (exam_score)

We begin by examining the distribution of our target variable. Understanding its shape helps us determine if transformations are needed to satisfy the normality assumptions of linear regression.

```
ggplot(train_data, aes(x = exam_score)) +
  geom_histogram(aes(y = ..density..), bins = 40, fill = "steelblue", color =
"white") +
  geom_density(color = "darkred", size = 1) +
  theme_minimal() +
  labs(title = "Distribution of Exam Scores",
       subtitle = "Density curve overlaid on histogram",
       x = "Exam Score",
       y = "Density")
```



Distribution of Final Exam Scores

Observation: The scores range from the mid-30s to 100, with a heavy left skew (a ceiling effect near 100). The mean is extremely high (~89), indicating a generally high-performing student body.

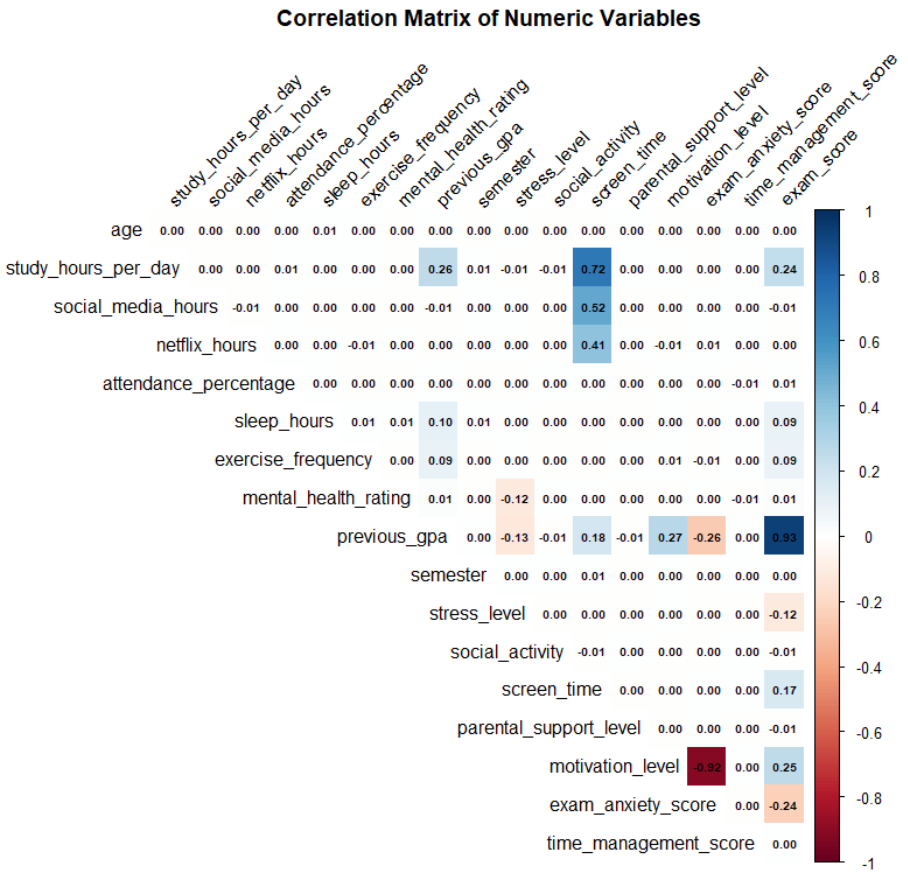
2.5.2 Correlation Matrix (Numeric Predictors)

To identify severe multicollinearity and find strong linear predictors, we generate a correlation matrix for all numeric variables.

```
# Select only numeric columns
numeric_data = train_data %>% select(where(is.numeric))

# Calculate Pearson correlation matrix
cor_matrix = cor(numeric_data, use = "complete.obs")

# Plot using corrplot
corrplot(cor_matrix, method = "color", type = "upper",
         tl.col = "black", tl.srt = 45, addCoef.col = "black",
         number.cex = 0.6, diag = FALSE,
         title = "Correlation Matrix of Numeric Variables",
         mar = c(0,0,1,0))
```



Key Takeaways:

1. **Strongest Predictor:** `previous_gpa` has an extraordinarily high correlation with `exam_score` ($r \approx 0.93$).
2. **Moderate Predictors:** `motivation_level` (0.25) and `study_hours_per_day` (0.24) show moderate positive correlations.
3. **Negative Predictors:** `exam_anxiety_score` (-0.24) and `stress_level` (-0.12) drag scores down.
4. **Multicollinearity:** There don't appear to be dangerously high correlations *between* the predictors themselves, which is good news for our baseline OLS model.

2.5.3 Bivariate Analysis: Strongest Numeric Predictors

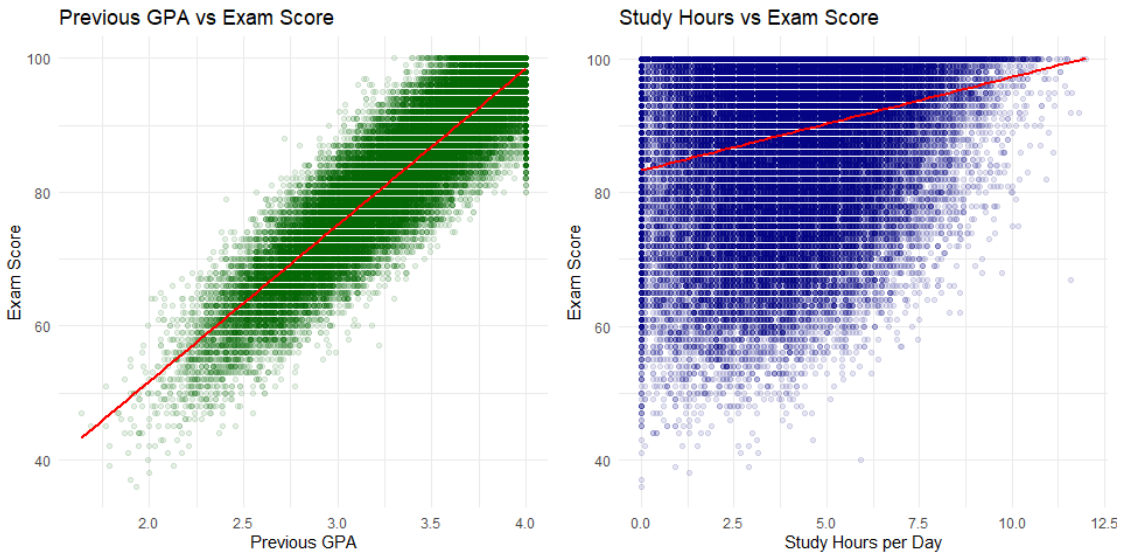
Let's visualize the relationship between our two strongest numeric predictors and the target variable using scatter plots with a smoothing line.

```
p1 = ggplot(train_data, aes(x = previous_gpa, y = exam_score)) +
  geom_point(alpha = 0.1, color = "darkgreen") + # Low alpha due to 64k rows
  geom_smooth(method = "lm", color = "red", se = FALSE) +
  theme_minimal() +
```

```
  labs(title = "Previous GPA vs Exam Score", x = "Previous GPA", y = "Exam
Score")

p2 = ggplot(train_data, aes(x = study_hours_per_day, y = exam_score)) +
  geom_point(alpha = 0.1, color = "navy") +
  geom_smooth(method = "lm", color = "red", se = FALSE) +
  theme_minimal() +
  labs(title = "Study Hours vs Exam Score", x = "Study Hours per Day", y =
"Exam Score")

grid.arrange(p1, p2, ncol = 2)
```



Observation: The relationship with `previous_gpa` is almost perfectly linear, confirming why it dominated the correlation matrix. `study_hours_per_day` also shows a clear positive, linear trend. This linearity suggests that parametric models (like OLS, Ridge, and LASSO) will perform very well.

2.5.4 Bivariate Analysis: Categorical Predictors

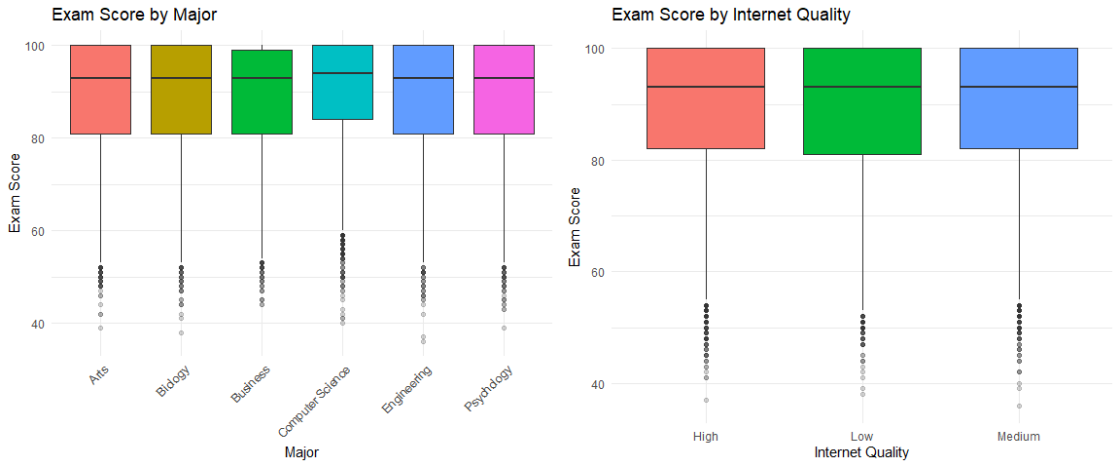
Finally, let's explore if certain categorical factors influence exam scores using boxplots.

```
p3 = ggplot(train_data, aes(x = major, y = exam_score, fill = major)) +
  geom_boxplot(outlier.alpha = 0.2) +
  theme_minimal() +
  theme(legend.position = "none", axis.text.x = element_text(angle = 45,
hjust = 1)) +
  labs(title = "Exam Score by Major", x = "Major", y = "Exam Score")

p4 = ggplot(train_data, aes(x = internet_quality, y = exam_score, fill =
internet_quality)) +
  geom_boxplot(outlier.alpha = 0.2) +
  theme_minimal() +
  theme(legend.position = "none") +
```

```
  labs(title = "Exam Score by Internet Quality", x = "Internet Quality", y =
"Exam Score")

grid.arrange(p3, p4, ncol = 2)
```



Observation: The median scores across different majors appear very similar, suggesting `major` might not be a massive driver of variance. However, `internet_quality` might show slight variations, emphasizing the need for our models to evaluate categorical dummy variables carefully.

---

3. Ordinary Least Squares (OLS) Regression

We establish our baseline by fitting a standard multiple linear regression using all available predictors.

```
ols_model = lm(exam_score ~ ., data = train_data)

# Multicollinearity check (Variance Inflation Factor)
vif_values = vif(ols_model)[,1:2]
vif_values
```

```
##                                  GVIF Df
## age                          1.000628  1
## gender                       1.001482  2
## major                        1.036985  5
## study_hours_per_day         13.100804  1
## social_media_hours           7.263731  1
## netflix_hours                5.012771  1
## part_time_job                1.000709  1
## attendance_percentage        1.000687  1
## sleep_hours                  1.013311  1
## diet_quality                 1.001621  2
## exercise_frequency           1.012414  1
```

```
## parental_education_level       1.002894  4
## internet_quality               1.001521  2
## mental_health_rating           1.015681  1
## extracurricular_participation  1.001061  1
## previous_gpa                   1.288494  1
## semester                       1.000495  1
## stress_level                   1.122801  1
## dropout_risk                   1.136473  1
## social_activity                1.000680  1
## screen_time                   23.048827  1
## study_environment              1.044879  4
## access_to_tutoring             1.021089  1
## family_income_range            1.001439  2
## parental_support_level         1.000503  1
## motivation_level               6.541087  1
## exam_anxiety_score             6.365959  1
## learning_style                 1.001818  3
## time_management_score          1.000794  1
```

```r
# Evaluation
ols_preds = predict(ols_model, newdata = test_data)
ols_rmse = RMSE(ols_preds, test_data$exam_score)
cat("OLS Test RMSE:", round(ols_rmse, 4), "\n")
```

```
## OLS Test RMSE: 4.174
```

---

4. Subset Selection (Stepwise)

To reduce model complexity, we use backward stepwise selection based on the Akaike Information Criterion (AIC).

```r
# trace = 0 hides the iterative output logs
stepwise_model = step(ols_model, direction = "backward", trace = 0)

stepwise_preds = predict(stepwise_model, newdata = test_data)
stepwise_rmse = RMSE(stepwise_preds, test_data$exam_score)
cat("Stepwise Test RMSE:", round(stepwise_rmse, 4), "\n")
```

```
## Stepwise Test RMSE: 4.1715
```

---

5. Regularization (Shrinkage Methods)

The `glmnet` package requires the input data to be formatted as matrices. This process automatically handles the conversion of categorical variables into dummy variables.

```r
# Prepare design matrices (removing the intercept column)
x_train = model.matrix(exam_score ~ ., train_data)[, -1]
```

```r
y_train = train_data$exam_score
x_test  = model.matrix(exam_score ~ ., test_data)[, -1]
y_test  = test_data$exam_score
```

5.1 Ridge Regression (L_2 Penalty)

Ridge regression shrinks coefficients to manage multicollinearity without entirely removing any variables.

```r
ridge_cv = cv.glmnet(x_train, y_train, alpha = 0)
best_lambda_ridge = ridge_cv$lambda.min

ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = best_lambda_ridge)
ridge_preds = predict(ridge_model, s = best_lambda_ridge, newx = x_test)
ridge_rmse = RMSE(ridge_preds, y_test)
cat("Ridge Test RMSE:", round(ridge_rmse, 4), "\n")
```

```
## Ridge Test RMSE: 4.2901
```

5.2 LASSO Regression (L_1 Penalty)

LASSO acts as an automated feature selector by forcing the coefficients of irrelevant variables exactly to zero.

```r
lasso_cv = cv.glmnet(x_train, y_train, alpha = 1)
best_lambda_lasso = lasso_cv$lambda.min

lasso_model = glmnet(x_train, y_train, alpha = 1, lambda = best_lambda_lasso)
lasso_preds = predict(lasso_model, s = best_lambda_lasso, newx = x_test)
lasso_rmse = RMSE(lasso_preds, y_test)
cat("LASSO Test RMSE:", round(lasso_rmse, 4), "\n")
```

```
## LASSO Test RMSE: 4.1723
```

5.3 Elastic Net (Hybrid)

Elastic net combines both L_1 and L_2 penalties.

```r
enet_cv = cv.glmnet(x_train, y_train, alpha = 0.5)
best_lambda_enet = enet_cv$lambda.min

enet_model = glmnet(x_train, y_train, alpha = 0.5, lambda = best_lambda_enet)
enet_preds = predict(enet_model, s = best_lambda_enet, newx = x_test)
enet_rmse = RMSE(enet_preds, y_test)
cat("Elastic Net Test RMSE:", round(enet_rmse, 4), "\n")
```

```
## Elastic Net Test RMSE: 4.1722
```

---

6. K-Nearest Neighbours (KNN) Regression

Given the computational intensity of calculating distance matrices for $n = 64{,}000$ training rows, we utilize a 5% stratified sample to train the KNN model. Center and scaling are applied to ensure distance metrics evaluate features fairly.

```
# Create a 5% stratified sample for KNN training
set.seed(2026)
knn_sample_idx = createDataPartition(train_data$exam_score, p = 0.05, list =
FALSE)
knn_train_data = train_data[knn_sample_idx, ]

# Train KNN with cross-validation
knn_model = train(
  exam_score ~ .,
  data = knn_train_data,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(k = c(5, 10, 15)),
  trControl = trainControl(method = "cv", number = 5)
)

knn_preds = predict(knn_model, newdata = test_data)
knn_rmse = RMSE(knn_preds, test_data$exam_score)
cat("KNN Test RMSE:", round(knn_rmse, 4), "\n")

## KNN Test RMSE: 8.1976
```

---

8. Final Performance Comparison

We compile the Root Mean Squared Errors (RMSE) of all applied techniques to determine the optimal predictive model. Lower RMSE indicates superior predictive accuracy.

```
# Compile Results
results = data.frame(
  Model = c("OLS Baseline", "Stepwise", "Ridge (L2)", "LASSO (L1)",
            "Elastic Net","KNN (Sampled)"),
  Test_RMSE = c(ols_rmse, stepwise_rmse, ridge_rmse, lasso_rmse,
                enet_rmse, knn_rmse)
)

# Sort and display the Comparison
final_compare = results %>%
  arrange(Test_RMSE) %>%
  mutate(Test_RMSE = round(Test_RMSE, 4))
```

```
kable(final_compare, caption = "Model Performance Comparison (Sorted by Test RMSE)")
```

Model Performance Comparison (Sorted by Test RMSE)

Model	Test_RMSE
Stepwise	4.1715
Elastic Net	4.1722
LASSO (L1)	4.1723
OLS Baseline	4.1740
Ridge (L2)	4.2901
KNN (Sampled)	8.1976